

CS202: Software Tools and Techniques for CSE

Lecture 7

Shouvick Mondal

shouvick.mondal@iitgn.ac.in

January 2025

Classwork: find the test-coverage map for *prog_v0.c*

```
1 // prog_v0.c
2 #include<stdio.h>
3 int g=5;
4 ...
5 int main()
6 {
7     int n, b;
8     scanf("%d", &n);
9     if (n >= 0)
10    {
11        if(n % 2 == 0)
12            g=n/2;
13        else
14            g=n/3;
15    }
16    else
17    {
18        n=g*2;
19    }
20    return 0;
21 }
```

```
1 // prog_v1.c
2 #include<stdio.h>
3 int g=7; //change 1
4 ...
5 int main()
6 {
7     int n, b;
8     scanf("%d", &n);
9     if (n >= 0)
10    {
11        if(n % 2 == 0)
12            g=n/4; //change 2
13        else
14            g=n/3;
15    }
16    else
17    {
18        n=g*2;
19    }
20    return 0;
21 }
```

8 test-cases: {2, -67, 3, 20, 40, 15, 34, -56}.

Classwork: find the test-coverage map for *prog_v0.c*

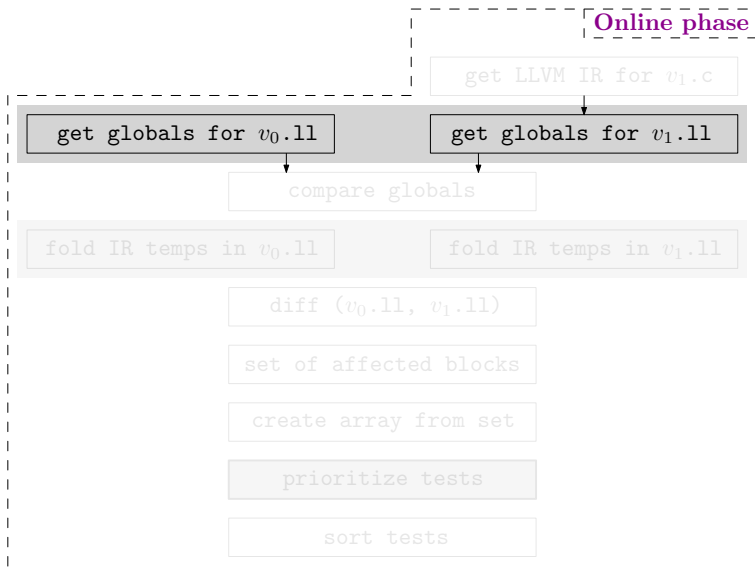
In terms of source-code line¹ coverage...

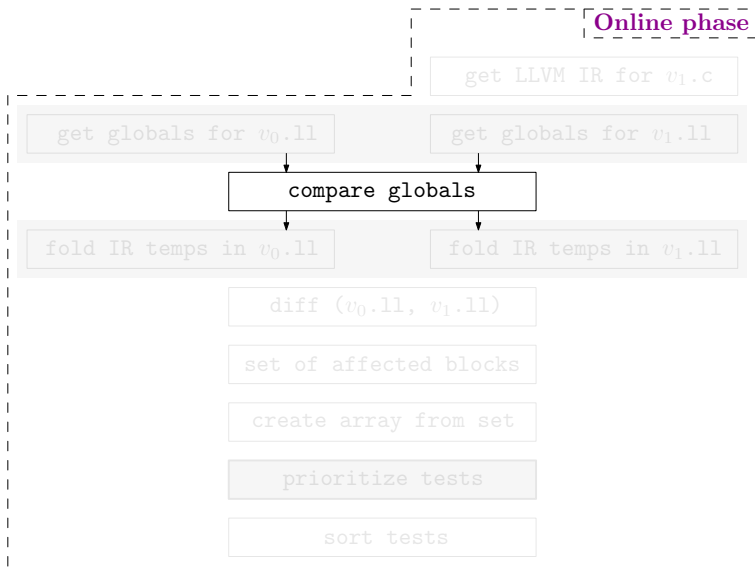
Forward map	t_1	→	l_3	l_7	l_8	l_9	l_{12}	l_{20}	(2)
	t_2	→	l_3	l_7	l_8	l_9	l_{18}	l_{20}	(-67)
	t_3	→	l_3	l_7	l_8	l_9	l_{14}	l_{20}	(3)
	t_4	→	l_3	l_7	l_8	l_9	l_{12}	l_{20}	(20)
	t_5	→	l_3	l_7	l_8	l_9	l_{12}	l_{20}	(40)
	t_6	→	l_3	l_7	l_8	l_9	l_{14}	l_{20}	(15)
	t_7	→	l_3	l_7	l_8	l_9	l_{12}	l_{20}	(34)
	t_8	→	l_3	l_7	l_8	l_9	l_{18}	l_{20}	(-56)

¹Only the interesting ones are shown.

Parallelizing online phase





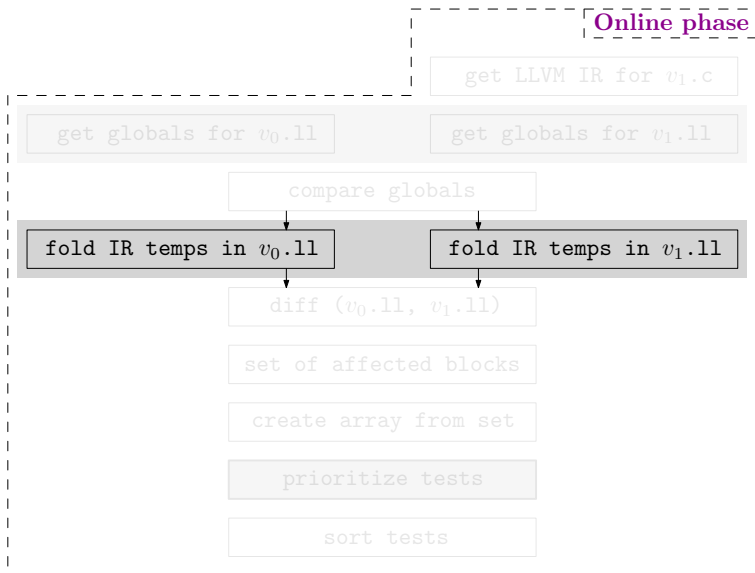


Determining changes: *globals₁* versus *globals₂*

```
1 globals1
2 ... @main() {
3 entry:
4   ...
5 if.then:
6   ...
7 if.else:
8   ...
9 if.end:
10  ...
11 }
12 ...

1 globals2
2 ... @main() {
3 entry:
4   ...
5 if.then: ; preds ...
6   ...
7   %1 = load i32, i32* %n, ...
8   %sub = sub nsw i32 %1, 32000
9   store i32 %sub, i32* %n, ...
10  ...
11 if.else: ; preds ...
12  ...
13 if.end: ; preds ...
14  ...
15 }
16 ...
```

Figure: Original program (*v0.ll*) and its new version (*v1.ll*).



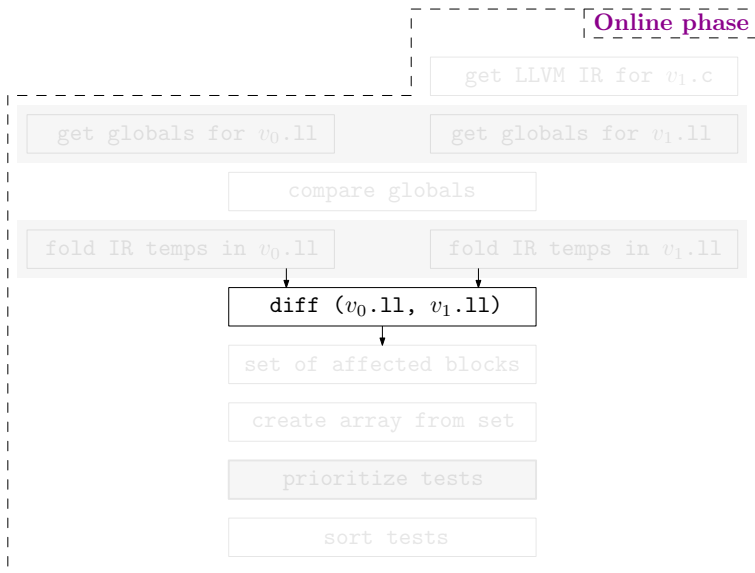
Folding of temporaries in LLVM IR

```
1 ...  
2 if.then: ; preds ...  
3 ...  
4 %1 = load i32, ...  
5 %sub = sub nsw i32 %1, ...  
6 ...  
7 ...
```

```
1 ...  
2 if.then: ; preds ...  
3 ...  
4 $ = load i32, ...  
5 %sub = sub nsw i32 $, ...  
6 ...  
7 ...
```

Figure: Folding of unnamed temporaries in LLVM IR.

Folding rule: regular expression `%[0-9][0-9]*` $\longrightarrow$ `$`.



Determining changes in basic blocks

```
1 globals1
2 ... @main() {
3 entry:
4 ...
5 if.then:
6 ...
7 if.else:
8 ...
9 if.end:
10 ...
11 }
12 ...

1 globals2
2 ... @main() {
3 entry:
4 ...
5 if.then: ; preds ...
6 ...
7 %1 = load i32, i32* %n, ...
8 %sub = sub nsw i32 %1, 32000
9 store i32 %sub, i32* %n, ...
10 ...
11 if.else: ; preds ...
12 ...
13 if.end: ; preds ...
14 ...
15 }
16 ...
```

Figure: Original program (v0.ll) and its new version (v1.ll).

Online phase

get LLVM IR for $v_1.c$

get globals for $v_0.ll$

get globals for $v_1.ll$

compare globals

fold IR temps in $v_0.ll$

fold IR temps in $v_1.ll$

diff ($v_0.ll$, $v_1.ll$)



set of affected blocks



create array from set

prioritize tests

sort tests

Creating set of affected basic blocks

- **set of affected blocks**: Scan filtered output of `diff` for `(function_name, block_name)` **pairs** and create **set of affected blocks**.
 - For our example: `{(main, if.then)}`.

Online phase

get LLVM IR for $v_1.c$

get globals for $v_0.ll$

get globals for $v_1.ll$

compare globals

fold IR temps in $v_0.ll$

fold IR temps in $v_1.ll$

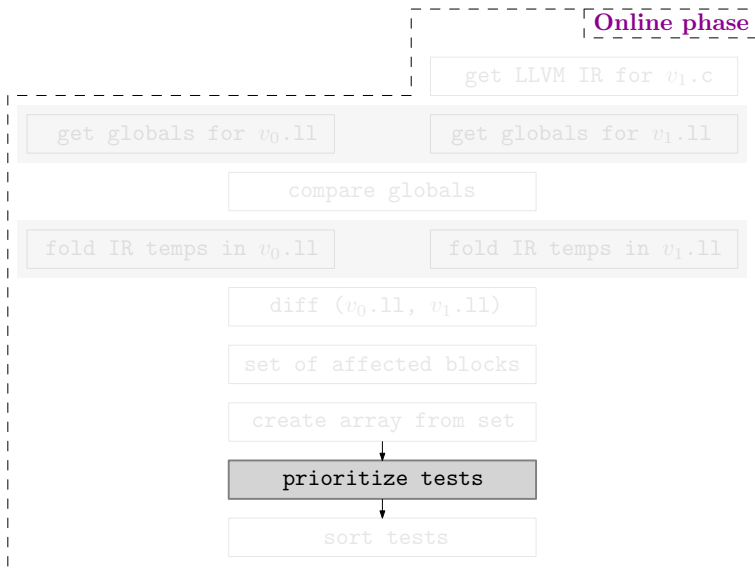
diff ($v_0.ll$, $v_1.ll$)

set of affected blocks

create array from set

prioritize tests

sort tests



Select and prioritize test-cases

Selected test-cases: {*test1*, *test3*, *test5*, *test7*, *test9*}

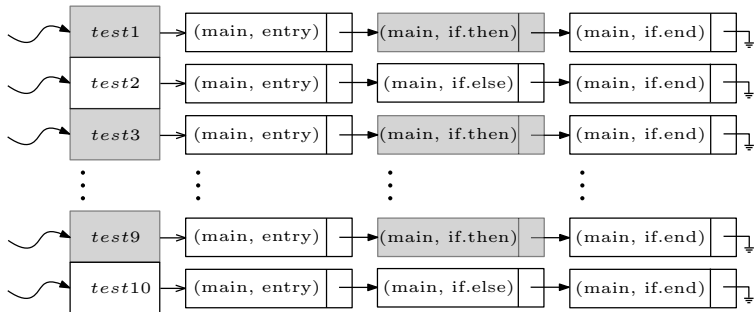


Figure: Test-case dependency graph.

Prioritization strategy

- *Relevance*: Run the test-case first, which covers **affected blocks as much as possible** without considering the coverage of additional basic blocks.

Prioritization strategy

- *Relevance*: Run the test-case first, which covers **affected blocks as much as possible** without considering the coverage of additional basic blocks.
- *Confinedness*: Run the test-case first, which covers affected blocks and which covers **as few as possible of the unaffected blocks**.

Prioritization strategy

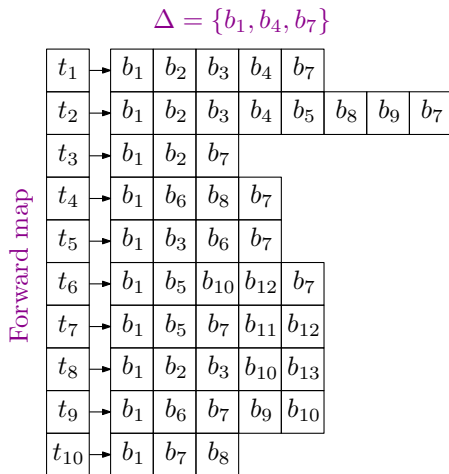
- *Relevance*: Run the test-case first, which covers **affected blocks as much as possible** without considering the coverage of additional basic blocks.
- *Confinedness*: Run the test-case first, which covers affected blocks and which covers **as few as possible of the unaffected blocks**.
- *Relevance and Confinedness*: Run the test-case first that **covers as much change as possible but still remains focused** to the potential fault-sites via shorter propagation paths.

Prioritization strategy

- *Relevance*: Run the test-case first, which covers **affected blocks as much as possible** without considering the coverage of additional basic blocks.
- *Confinedness*: Run the test-case first, which covers affected blocks and which covers **as few as possible of the unaffected blocks**.
- *Relevance and Confinedness*: Run the test-case first that **covers as much change as possible but still remains focused** to the potential fault-sites via shorter propagation paths.

In case of priority collisions, test-cases are applied in the order they appear in the test-suite.

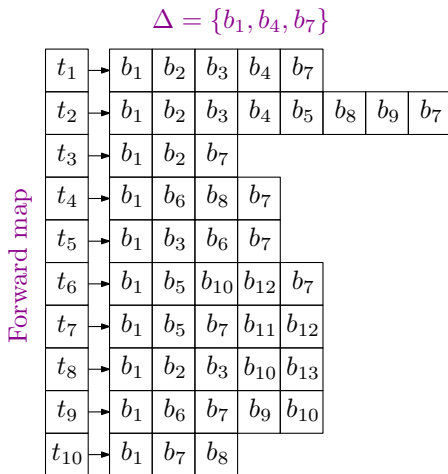
Example prioritization: unordered



$\langle t_1, t_2, t_3, t_4, t_5, t_6, t_7, t_8, t_9, t_{10} \rangle$

Unordered

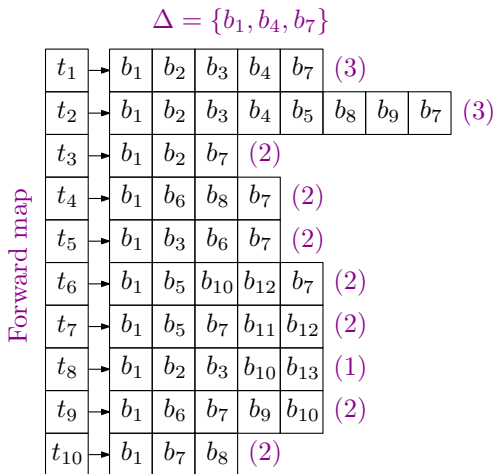
Example prioritization: relevance



Prioritized by relevance

$$rel(t) = |cov(t) \cap \Delta|$$

Example prioritization: relevance

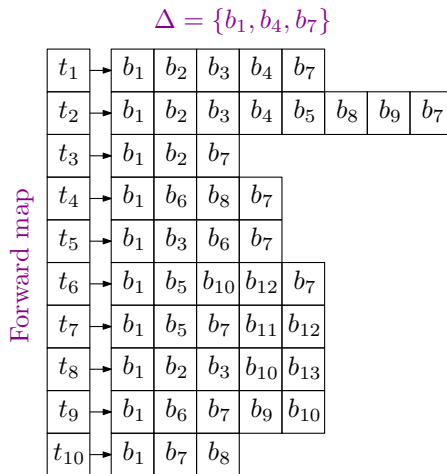


$\langle t_1, t_2, t_3, t_4, t_5, t_6, t_7, t_9, t_{10}, t_8 \rangle$

Prioritized by relevance

$$rel(t) = |cov(t) \cap \Delta|$$

Example prioritization: confinedness



Prioritized by confinedness

$$con(t) = |cov(t) - \Delta|^{-1}$$

Example prioritization: confinedness

$\Delta = \{b_1, b_4, b_7\}$

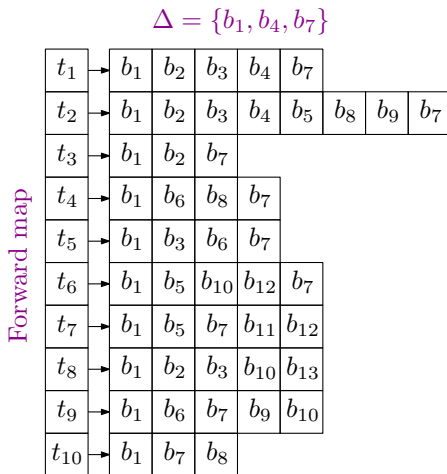
Forward map	t_1	→	b_1	b_2	b_3	b_4	b_7	(0.5)				
	t_2	→	b_1	b_2	b_3	b_4	b_5	b_8	b_9	b_7	(0.2)	
	t_3	→	b_1	b_2	b_7	(1)						
	t_4	→	b_1	b_6	b_8	b_7	(0.5)					
	t_5	→	b_1	b_3	b_6	b_7	(0.5)					
	t_6	→	b_1	b_5	b_{10}	b_{12}	b_7	(0.34)				
	t_7	→	b_1	b_5	b_7	b_{11}	b_{12}	(0.34)				
	t_8	→	b_1	b_2	b_3	b_{10}	b_{13}	(0.25)				
	t_9	→	b_1	b_6	b_7	b_9	b_{10}	(0.34)				
	t_{10}	→	b_1	b_7	b_8	(1)						

$\langle t_3, t_{10}, t_1, t_4, t_5, t_6, t_7, t_9, t_8, t_2 \rangle$

Prioritized by confinedness

$$con(t) = |cov(t) - \Delta|^{-1}$$

Example prioritization: relevance and confinedness



Prioritized by relevance and confinedness

$$relcon(t) = |cov(t) \cap \Delta| + |cov(t) - \Delta|^{-1}$$

Example prioritization: relevance and confinedness

$\Delta = \{b_1, b_4, b_7\}$

Forward map	t_1	→	b_1	b_2	b_3	b_4	b_7	(3.5)			
	t_2	→	b_1	b_2	b_3	b_4	b_5	b_8	b_9	b_7	(3.2)
	t_3	→	b_1	b_2	b_7	(3)					
	t_4	→	b_1	b_6	b_8	b_7	(2.5)				
	t_5	→	b_1	b_3	b_6	b_7	(2.5)				
	t_6	→	b_1	b_5	b_{10}	b_{12}	b_7	(2.34)			
	t_7	→	b_1	b_5	b_7	b_{11}	b_{12}	(2.34)			
	t_8	→	b_1	b_2	b_3	b_{10}	b_{13}	(1.25)			
	t_9	→	b_1	b_6	b_7	b_9	b_{10}	(2.34)			
	t_{10}	→	b_1	b_7	b_8	(3)					

$\langle t_1, t_2, t_3, t_{10}, t_4, t_5, t_6, t_7, t_9, t_8 \rangle$

Prioritized by relevance and confinedness

$$relcon(t) = |cov(t) \cap \Delta| + |cov(t) - \Delta|^{-1}$$

Online phase

get LLVM IR for $v_1.c$

get globals for $v_0.ll$

get globals for $v_1.ll$

compare globals

fold IR temps in $v_0.ll$

fold IR temps in $v_1.ll$

diff ($v_0.ll$, $v_1.ll$)

set of affected blocks

create array from set

prioritize tests

↓
sort tests

Sorting after selection and award-valuation

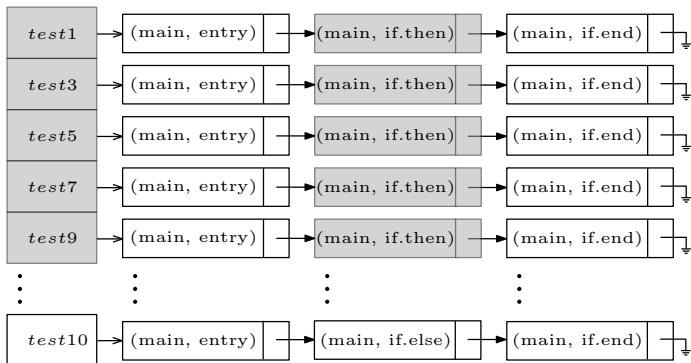


Figure: Sorted test-case dependency graph.

Classwork: regression test selection

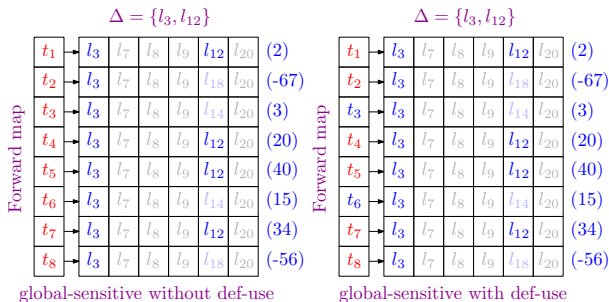
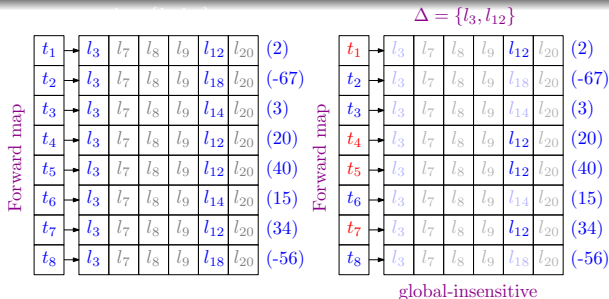
```
1 // prog_v0.c
2 #include<stdio.h>
3 int g=5;
4 ...
5 int main()
6 {
7     int n, b;
8     scanf("%d", &n);
9     if (n >= 0)
10    {
11        if(n % 2 == 0)
12            g=n/2;
13        else
14            g=n/3;
15    }
16    else
17    {
18        n=g*2;
19    }
20    return 0;
21 }
```

```
1 // prog_v1.c
2 #include<stdio.h>
3 int g=7; //change 1
4 ...
5 int main()
6 {
7     int n, b;
8     scanf("%d", &n);
9     if (n >= 0)
10    {
11        if(n % 2 == 0)
12            g=n/4; //change 2
13        else
14            g=n/3;
15    }
16    else
17    {
18        n=g*2;
19    }
20    return 0;
21 }
```

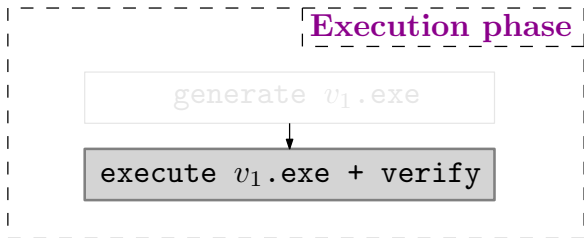
8 test-cases: {2, -67, 3, 20, 40, 15, 34, -56}.

- Select test-cases when analysis is: (i) **global-insensitive**,
(ii) **global-sensitive** without *def-use*, and (iii) **global-sensitive** with *def-use*.

Classwork: regression test selection (more precise?)



Parallelizing execution phase



Execute selected tests

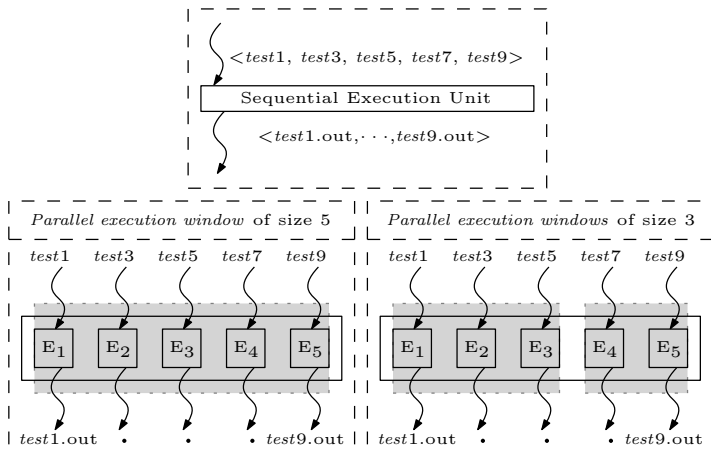


Figure: Executing selected tests: sequential versus parallel windows (shaded in gray).

Parallelization windows

- Sequential test execution follows the observable test timeline.
- Parallel execution window (PEW): Test-execution is multi-threaded by slicing the timeline using a groups of threads called *windows* (ω).

Parallelization windows

- Sequential test execution follows the observable test timeline.
- Parallel execution window (PEW): Test-execution is multi-threaded by slicing the timeline using a groups of threads called *windows* (ω).
- A PEW ω is said to have failed if there exists a failed test-case $t_f \in \omega$.

Parallelization windows

- Sequential test execution follows the observable test timeline.
- Parallel execution window (PEW): Test-execution is multi-threaded by slicing the timeline using a groups of threads called *windows* (ω).
- A PEW ω is said to have failed if there exists a failed test-case $t_f \in \omega$.
- Parallel μ window ($P_\mu W$): Generalized parallel window. $\mu \in \{\text{coverage-collection, map-population, award-valuation}\}$.

Parallelization windows

- Sequential test execution follows the observable test timeline.
- Parallel execution window (PEW): Test-execution is multi-threaded by slicing the timeline using a groups of threads called *windows* (ω).
- A PEW ω is said to have failed if there exists a failed test-case $t_f \in \omega$.
- Parallel μ window ($P_\mu W$): Generalized parallel window. $\mu \in \{\text{coverage-collection, map-population, award-valuation}\}$.
- Parallelism can be tuned by changing the window size $|\omega|$.

- **Parallelizes** different phases of regression testing.

Summary of Mahtab

- **Parallelizes** different phases of regression testing.
- **Platform-independent** analysis leveraging LLVM IR.

Summary of Mahtab

- **Parallelizes** different phases of regression testing.
- **Platform-independent** analysis leveraging LLVM IR.
- **Prioritization** coupled with **multi-threaded** windows.

Summary of Mahtab

- **Parallelizes** different phases of regression testing.
- **Platform-independent** analysis leveraging LLVM IR.
- **Prioritization** coupled with **multi-threaded** windows.
- **Failure-aware** but fault-blind strategy.

- **Parallelizes** different phases of regression testing.
- **Platform-independent** analysis leveraging LLVM IR.
- **Prioritization** coupled with **multi-threaded** windows.
- **Failure-aware** but fault-blind strategy.

Thank You

Paper: <https://doi.org/10.1016/j.jss.2019.110403>

Artifacts: <https://data.mendeley.com/datasets/7fvwj88jvm/2>